Chapter 12 — Networked Programs (Sockets in Python)

1. Notes (Concept Explanation)

Network Ki Layered Architecture

Network ek **layered architecture** pe kaam karta hai (jese TCP/IP stack). Har layer ka apna kaam hota hai:

- **Link Layer** — Ek hop tak data pohanchane ka kaam karti hai (jese router se router).
- **Internet Layer** — Multiple hops (15-20 tak) ke through packets ko source se destination tak route karti hai.
- **Transport Layer** — Yahan se hum start karte hain. Ye layer ye assume karti hai ke do computers ke darmiyan ek **reliable pipe/connection** ban chuki hai — jese phone call. Hum lower layers (link, internet) ko ignore karte hain kyunke transport layer unko abstract kar deti hai.

> Jab tum browser mein URL open karte ho, request 15-20 hops se guzarti hui server tak jati hai, server data process karke wapis 15-20 hops se hoke tumhare paas aata hai — yehi cheez tumhe webpage ki shakal mein dikhti hai.

Socket Kya Hota Hai?

- Do processes (ek tumhare computer pe, ek server pe) internet ke through connect hote hain — is connection ko **socket** kehte hain.
- Socket ek **bi-directional pipe** hai — dono taraf se data bhej aur receive kar sakte ho (talk + listen).
- Analogy: Ye ek "data phone call" hai application se application ke darmiyan.
  - Client side: Browser (Chrome, Firefox, Internet Explorer)
  - Server side: Web server (IIS, Apache, Tomcat)
- Server generally **hamesha up rehta hai**, aur client jab chahe request bhej sakta hai.

Port Numbers — Extensions Ka Concept

- Ek organization ka ek phone number hota hai, aur us pe "extension" enter karke specific banda milta hai — **ports bilkul yehi kaam karte hain**.
- Server pe multiple services chal rahi hoti hain, har service ek specific **port number** pe "extension" ki tarah sunti hai.
- Convention ke mutabiq standard ports:

Service	Port
Email (SMTP)	25
Telnet (insecure login)	23
HTTP (web)	80
HTTPS (secure web)	443

- Agar URL mein `:number` dikhe (jese `example.com:8080`), iska matlab web server default port (80/443) ki bajaye kisi custom port pe chal raha hai.

Python Mein Socket Banane Ka Tareeqa

```python
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
```

Is code ki breakdown:

1. `import socket` — socket library import karo (Python built-in hai, lekin use karne se pehle import zaroori hai).

2. `socket.socket(...)` — Socket object banata hai. Ye ek **unopened/half file handle** ki tarah hai — outward-looking, lekin abhi tak kisi se connect nahi.
    - `AF_INET` → internet ke across kaam karega
    - `SOCK_STREAM` → **stream socket**, matlab data characters ki continuous series ki tarah aata hai (blocks ki bajaye)
3. `mysock.connect((host, port))` — Ye actual "phone call" hai: given host (`data.pr4e.org`) ke given port (`80`) pe connection banane ki koshish karta hai.
    - Ye line **fail/blow up** ho sakti hai agar us port pe koi service listen nahi kar rahi, network connection nahi, ya server down hai — ye ek legitimate/normal scenario hai jo handle karna padta hai.
4. Is point tak koi data send nahi hua — sirf connection establish hui hai ("phone uthaya gaya"). Agla step decide karna hoga ke kya data send/receive karna hai.

Key Insight

Sirf **3 lines of Python** mein hum us poore internet infrastructure ko use kar rahe hain jo lakhon engineers ne decades mein banaya — yehi wajah hai ke Python itna popular hai (simplicity + abstraction ka power).

---

2. Active Recall Question Bank

1. Transport layer kis cheez ko "assume" kar leti hai jo lower layers handle karti hain?
2. Link layer aur Internet layer mein kya farq hai?
3. Socket ki simple tareeqe se definition do.
4. Socket ko "bi-directional" kyun kaha jata hai?
5. Client aur server side pe kaunse example applications chalte hain?
6. Port numbers ko phone extensions se compare kyun kiya gaya?
7. HTTP aur HTTPS ke default port numbers kya hain?
8. Telnet aur Email (SMTP) ke default ports kya hain?
9. Agar URL mein `:8080` jesa kuch dikhe to iska matlab kya hai?

10. `socket.socket()` call karne se socket "connected" ho jata hai — sahi ya ghalat? Wajah batao.
11. `AF_INET` aur `SOCK_STREAM` parameters ka kya matlab hai?
12. `mysock.connect((host, port))` line fail kyun ho sakti hai?
13. Stream socket ka data doosre socket types se kaise different hota hai?
14. Ek webpage load hone mein packets kitni baar network cross karte hain (roughly)?
15. Kyun kaha gaya ke Python mein sirf 3 lines se hum "half a million engineers" ka kaam use kar rahe hain?

---

3. Answer Key

1. Transport layer ye assume kar leti hai ke ek **reliable pipe/connection** do computers ke darmiyan already ban chuki hai — lower layers (link, internet) ki complexity ko ignore kar deti hai.
2. Link layer ek single hop tak data pohanchati hai; Internet layer multiple hops (~15-20) ke through packets ko end-to-end route karti hai.
3. Socket do processes (ek local, ek remote) ke darmiyan ek connection hai jo internet ke through banti hai, jisse dono talk aur listen kar sakte hain.
4. Kyunke dono taraf se data bhej aur receive kiya ja sakta hai — jese phone call mein dono log bol aur sun sakte hain.
5. Client side: browser (Chrome, Firefox, IE). Server side: web server software (IIS, Apache, Tomcat).
6. Kyunke ek organization ka ek phone number hota hai aur extension se specific banda milta hai — similarly server ka ek address hota hai aur port se specific service/process milta hai.
7. HTTP → 80, HTTPS → 443.
8. Telnet → 23, Email (SMTP) → 25.
9. Iska matlab hai web server default port (80 ya 443) ki bajaye kisi custom/non-standard port pe run ho raha hai.

10. **Ghalat.** `socket.socket()` sirf ek unopened/unconnected socket object banata hai (jese half file handle) — actual connection `connect()` call se banti hai.
11. `AF_INET` batata hai ke socket internet address family use karega; `SOCK_STREAM` batata hai ke ye ek stream-type socket hai jahan data continuous character stream ki tarah aata hai, block-wise nahi.
12. Kyunke ho sakta hai us specified port pe koi service listen na kar rahi ho, network connection na ho, ya poora server hi down ho — ye sab legitimate real-world scenarios hain.
13. Stream socket data ko ek continuous series of characters ki tarah deta hai (ek ke baad ek), jabke doosre types (jese datagram) data blocks/chunks mein deal karte hain aur handle karna zyada mushkil hota hai.
14. Roughly 15-20 baar (request jaate hue aur response wapis aate hue dono taraf).
15. Kyunke Python ki socket library ne underlying complex network protocols (jo decades mein lakhon engineers ne design kiye) ko itna abstract kar diya hai ke sirf `import socket`, `socket.socket()`, aur `.connect()` — 3 lines se ek working internet connection ban jata hai.

Part 2 — Application Protocol, HTTP, aur Request-Response Cycle

1. Notes (Concept Explanation)

Application Protocol Kya Hota Hai?

- Socket sirf ek **connection/endpoint** banata hai — `connect()` se hum kisi far host (jese `www.py4e.org`) ke kisi port (jese 80) tak pohanch jate hain.
- Lekin sirf connect hone se kaam nahi chalta — ab ye decide karna hota hai ke us connection pe **kya bhejna hai aur kya expect karna hai**. Yehi rules **application protocol** kehlate hain.
- Analogy: Phone call connect hone ke baad "Hello" kehna ek protocol hai — dono taraf ko pata hona chahiye ke pehle kaun bolega aur kya bolega.

- Application protocol ka kaam: decide karna ke **kaun pehle "baat" karega, kya kahega, aur agar doosri taraf jawab na de to kya hoga.**

HTTP — Sab Se Common Application Protocol

- Is course mein hum jo protocol use karenge wo hai **HTTP (HyperText Transfer Protocol)** — simple aur widely-used hai.
- Lekin HTTP akela protocol nahi — mail, file transfer, remote login jese kaam ke liye alag-alag application protocols hote hain. Sab ki base **socket** hoti hai, lekin har ek ke apne "rules of the road" hain (jese traffic mein left ya right side drive karna — koi bhi convention chale, bas sab agree hona chahiye).

URL — Web Ka Genius Idea

- URL (Uniform Resource Locator) mein 3 cheezein encode hoti hain:
  1. **Protocol** (jese http, ftp)
  2. **Host** (kis server se baat karni hai)
  3. **Document/resource** (kya retrieve karna hai)
- Ye 1990s mein bohat revolutionary tha, kyunke pehle har protocol (FTP, Telnet, SMTP) ke liye alag program use karna padta tha aur manually host se connect karna padta tha.
- HTTP ko **Tim Berners-Lee aur Robert Cailliau** ne **1989-1990** mein **CERN** mein invent kiya tha.

Request-Response Cycle (Click Se Page Load Tak)

Jab user kisi webpage pe link pe click karta hai, ye sequence hoti hai:

1. **Browser (client app)** link ka click detect karta hai.
2. Browser server (jese `drchuck.com`) ke **Port 80** pe connection banata hai.
3. Browser ek precisely-specified **request** bhejta hai (standard format follow karke).
4. **Server** apna kaam karta hai — files read karta hai, code run karta hai, aur ek response construct karta hai.

5. Server response wapis bhejta hai — usually **HTML** (HyperText Markup Language) format mein.
    - Yaad rahe: **HTTP = protocol** (baat karne ka tareeqa), **HTML = format** (document ka structure) — dono different cheezein hain.
6. Browser HTML/CSS/JS ko parse karta hai aur ek "pretty" webpage render karta hai.
7. Agar us page mein aage koi link ho, to click karne pe ye poora cycle **dobara** hota hai.

Ye poora process **Request-Response Cycle** kehlata hai.

Internet Standards aur RFC

- HTTP jese protocols standard documents pe based hain jo **IETF (Internet Engineering Task Force)** publish karti hai.
- In documents ko **RFC (Request For Comments)** kaha jata hai — naam thora ironic hai kyunke decades purane established protocols abhi bhi "RFC" (comments ke liye request) kehlate hain, matlab community hamesha improvement ke liye open rehti hai.
- Example: **RFC 2616** — HTTP/1.1 ko define karta hai. Isme exact rules likhe hote hain, jese request ka format: `METHOD [space] URI [space] HTTP-VERSION [carriage-return][line-feed]`.
    - Yahan spacing bhi strict hai — do spaces use karoge to protocol fail ho jayega.

Telnet Se Manually HTTP Request Bhejna

- **Telnet** ek purana program hai jo kisi host ke kisi port pe raw socket connection khol deta hai — jo bhi type karo wo seedha server ko bhej diya jata hai.
- Aajkal login ke liye use nahi hota kyunke ye data ko **encrypt nahi karta** (insecure), lekin sockets samajhne ke liye acha demonstration tool hai.
- Command: `telnet www.py4e.org 80` — is se manually port 80 pe connect hoke, hath se HTTP request type karke server se response le sakte ho.

- Important: Request **fast type karni padti hai** aur end mein **do baar Enter** (ek blank line) dena zaroori hai — warna server "human" samajh kar request ignore/reject kar sakta hai (kyunke servers programs se baat karne ke liye designed hote hain, slow-typing humans se nahi).
- Response mein pehle **headers** (document ke baare mein metadata) aate hain, phir ek blank line, phir **actual document**, aur phir connection close ho jata hai.

Command Line Aur Hacking Ka Connection

- Command line se protocols ko directly bolna seekhna important hai — yehi tareeqa hai jaise **real hackers** kaam karte hain: wo GUI ki "pretty clicky buttons" ke peeche jo raw protocol chal raha hota hai usay directly samajhte aur use karte hain.
- Secure servers ab pehle jitna asaan hack karna nahi (challenge-response mechanisms ki wajah se), lekin underlying protocols samajhna developer/hacker dono ke liye foundational skill hai.

Aage Kya?

Agla step: Yehi HTTP request jo humne manually telnet se type ki, wo ab **Python code se automatically bhejni hai** — yani ek simple **Python web browser** likhna.

---

2. Active Recall Question Bank (Part 2)

1. Application protocol kis cheez ko decide karta hai (socket connect hone ke baad)?
2. HTTP full form aur iska basic role kya hai?
3. URL mein kaunsi 3 cheezein encode hoti hain?
4. URL invent hone se pehle log different protocols (FTP, Telnet, SMTP) kaise use karte the?
5. HTTP ko kis ne aur kahan invent kiya, aur kab?
6. HTTP aur HTML mein basic farq kya hai?
7. Request-Response cycle ke 7 steps summarize karo (click se render tak).
8. IETF aur RFC kya hain?
9. RFC 2616 kis cheez ko define karta hai?

10. HTTP request ka basic format (method, URI, version) kya hota hai, aur spacing kyun important hai?
11. Telnet kya hai aur is se server se manually kaise baat ki ja sakti hai?
12. Telnet se HTTP request bhejte waqt kya dhyan rakhna zaroori hai (2 cheezein)?
13. Server response mein kya-kya include hota hai (order mein)?
14. Telnet ko login ke liye kyun avoid kiya jata hai aajkal?
15. Command line samajhna hacking/security perspective se kyun important hai?

---

3. Answer Key (Part 2)

1. Application protocol decide karta hai ke connection pe **kya data bheja jayega aur kya receive expect kiya jayega** — yani "rules of the road" (kaun pehle bolega, kya bolega, no-response pe kya hoga).
2. **HyperText Transfer Protocol** — ye web documents (pages) ko retrieve karne ka standard, simple protocol hai.
3. **Protocol** (jese http), **Host** (jese www.py4e.org), aur **Document/resource** (jo retrieve karna hai).
4. Har protocol (FTP, Telnet, SMTP) ke liye alag program use karna padta tha, aur manually us host se sahi tareeqe se connect karna padta tha — koi unified system nahi tha.
5. **Tim Berners-Lee aur Robert Cailliau** ne, **CERN** mein, **1989-1990** mein invent kiya.
6. **HTTP** communication ka **protocol** hai (baat karne ka tareeqa), jabke **HTML** us document ka **format/structure** hai jo response mein milta hai.
7. (1) Browser click detect karta hai → (2) Server ke port 80 pe connect karta hai → (3) Request bhejta hai → (4) Server process karke response banata hai → (5) Server HTML response bhejta hai → (6) Browser HTML/CSS/JS parse karke render karta hai → (7) Agla link click hone pe cycle repeat hoti hai.
8. **IETF (Internet Engineering Task Force)** wo group hai jo internet standards banata hai; **RFC (Request For Comments)** un standards ke official documents hain.

9. RFC 2616 **HTTP version 1.1** ko define karta hai — request/response ka exact format aur rules.
10. Format: `METHOD [space] URI [space] HTTP-VERSION [CRLF]`. Spacing strict hai kyunke ye protocol specification ka hissa hai — extra ya missing space se request fail ho sakti hai (parser confuse ho jata hai).
11. Telnet ek program hai jo kisi host ke kisi port pe raw socket khol deta hai; jo bhi keyboard se type karo wo seedha us server ko bhej diya jata hai — is tarah manually HTTP request type karke server se raw response dekha ja sakta hai.
12. (1) Request **fast type** karni chahiye (warna server human samajh kar reject kar sakta hai), (2) End mein **do baar Enter** (blank line) dena zaroori hai.
13. Pehle **headers** (metadata about the document), phir ek **blank line**, phir **actual document/content**, phir connection **close** ho jata hai.
14. Kyunke Telnet data ko **encrypt nahi karta** — isliye ye insecure hai aur credentials/data plaintext mein interceptable hote hain.
15. Kyunke real hackers GUI ki "pretty" layer ke peeche jo raw protocol communication ho rahi hoti hai usay directly samajhte aur manipulate karte hain — command line se protocols ko directly bolna unhe developers se better system understanding deta hai.

Part 3 — Python Mein Web Browser Likhna

1. Notes (Concept Explanation)

Poora Flow — Socket Se Le Kar Data Print Tak

Ab hum wahi kaam jo pehle Telnet se **hath se** kiya tha, usay **Python code** se automate karenge. Poora program bas ~**10 lines** ka hai.

```python
```

```python
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)

while True:
    data = mysock.recv(512)
    if len(data) < 1:
        break
    print(data.decode())

mysock.close()
```

Step-by-Step Breakdown

1. **Socket banana aur connect karna** (jo pichle sections mein cover ho chuka):
   - `socket.socket(socket.AF_INET, socket.SOCK_STREAM)` → stream-based socket banata hai jo internet ke across chal sakta hai.
   - `mysock.connect(('data.pr4e.org', 80))` → server ko "phone call" karta hai. Agar us port pe koi software listen nahi kar raha to ye **blow up (fail)** ho jayega.
2. **"Kaun pehle bolega?" — Application Protocol Ka Decision**
   - Connection banne ke baad, web server **kuch nahi bolta** — wo chup rehta hai aur client (browser) ka wait karta hai.
   - Isliye **hum (client) pehle baat karte hain** — ye HTTP protocol ka rule hai.
3. **GET Request Banana Aur Bhejna**
   - Request ek **string** ki tarah likhi jati hai: `GET <path> HTTP/1.0` + do newlines (blank line zaroori hai — jese pichle section mein telnet se manually type karte waqt tha).

- `.encode()` — Ye string ko **bytes** mein convert karta hai, kyunke network pe data bytes ki form mein travel karta hai (raw text ki tarah nahi). Isay **"encoding"** kehte hain.
- `mysock.send(cmd)` — Ye encoded request ko socket ke through actual network pe bhej deta hai.

4. **Server Ka Response**
    - Server request receive karke apna kaam karta hai (file read, code run, etc.) aur phir data wapis bhejna start karta hai.
5. **Data Receive Karne Ka Loop**
    - `mysock.recv(512)` — Ek time mein **512 characters/bytes** receive karta hai (chunks mein, pura data ek sath nahi aata).
    - Loop tab tak chalta hai jab tak socket band na ho jaye.
    - **Agar `recv()` se 0 bytes milein (`len(data) < 1`), iska matlab stream/connection close ho gaya hai** — ye end-of-data ka signal hai (jese Telnet session mein "connection closed" dikhta tha).
    - Agar abhi connection open hai lekin data slow aa raha hai, to `recv()` **wait** karega (pause). Slow network pe ye pause noticeable hota hai; fast network pe itni jaldi hota hai ke pata hi nahi chalta.
6. **Decode Karna Zaroori Kyun Hai?**
    - Data jo outside world (network) se aata hai, wo **bytes** ki form mein hota hai — usay print karne se pehle `.decode()` se wapis **string/text** mein convert karna padta hai.
7. **Loop Khatam, Socket Close**
    - Jab data khatam ho jaye (0 bytes mil jayein), loop `break` ho jata hai aur `mysock.close()` se connection band kar di jati hai.

Output Kaise Dikhta Hai?

- Bilkul waisa hi jaisa Telnet se manually karne pe dikhta tha:

1. Pehle **headers** (metadata — jese ye batata hai ke response mein aage kya type ka content aa raha hai).
2. Ek **blank line** — ye signal hai ke headers khatam, ab actual data/content start ho raha hai.
3. Phir **actual content** (is example mein `romeo.txt` file ka text).

Important Caveat — Sirf Text Ke Liye Kaam Karta Hai (Is Form Mein)

- Ye simple program **sirf text data** (jese `.txt` file) ke liye theek se kaam karta hai.
- Agar isi code se koi **image (jpeg/png)** retrieve karne ki koshish ki jaye, to output **gibberish/garbage characters** dikhega — kyunke binary data (images) ko simple `.decode()` se readable text mein convert nahi kiya ja sakta. (Isay properly handle karne ka tareeqa aage discuss hoga.)

Next Topic Ka Preview

Aage ye dekha jayega ke jab characters computer ke **andar (strings)** se **socket ke through bahar (network)** jate hain aur wapis aate hain, to unke saath **encoding/decoding** level pe exactly kya hota hai.

---

2. Active Recall Question Bank (Part 3)

1. Python mein poora simple web browser kitni lines ka ban jata hai?
2. Connection banne ke baad web server pehle kuch bolta hai ya client? Kyun?
3. GET request bhejne se pehle `.encode()` kyun call kiya jata hai?
4. `mysock.send(cmd)` ka kaam kya hai?
5. `mysock.recv(512)` mein `512` ka kya matlab hai?
6. Loop mein `if len(data) < 1: break` line ka kya matlab/purpose hai?
7. Agar network slow ho to `recv()` call ka behavior kaisa hoga?
8. `.decode()` ki zaroorat kyun parti hai response data pe?
9. Server response mein headers aur actual data ke darmiyan kaunsi cheez separator ka kaam karti hai?

10. Ye simple program agar image (jpeg/png) retrieve kare to kya hoga aur kyun?
11. GET request string mein do newlines (`\r\n\r\n`) kyun zaroori hain?
12. Socket connect() call kab "blow up" (fail) ho sakti hai?
13. Poora request-response cycle is program mein kin steps mein hota hai (short list)?

---

3. Answer Key (Part 3)

1. Sirf ~**10 lines** ka.
2. **Client (browser)** pehle bolta hai — web servers connection ke baad chup rehte hain aur client ka wait karte hain; ye HTTP application protocol ka established rule hai.
3. Kyunke network pe data **bytes** ki form mein bhejna hota hai, jabke Python string normally text/unicode hoti hai — `.encode()` string ko bytes mein convert karta hai taake wo network-transmittable ho.
4. Ye encoded request (bytes) ko socket ke through actual network par server tak bhejta hai.
5. `512` batata hai ke ek `recv()` call mein **maximum kitne bytes/characters ek dafa mein receive kiye jayein** — data chunks mein aata hai, pura ek sath nahi.
6. Ye check karta hai ke agar receive hui data ki length 0 ho (matlab kuch data nahi mila), to iska matlab connection/stream **close ho chuka hai** — is condition pe loop `break` ho kar khatam ho jata hai.
7. `recv()` **wait/pause** karega jab tak naya data na aaye — slow network pe ye pause visible hota hai (agar print statements ho), fast network pe itni jaldi ho jata hai ke notice nahi hota.
8. Kyunke network se aane wala data **raw bytes** ki form mein hota hai — usay human-readable text/string mein print karne se pehle `.decode()` se convert karna zaroori hai.
9. Ek **blank line** — ye headers (metadata) aur actual content ke darmiyan separator ka kaam karti hai.

10. Output **gibberish/garbage** dikhega, kyunke image binary data hai aur `.decode()` (jo text ke liye hai) us binary data ko sahi tarah text mein convert nahi kar sakta.
11. Ek newline HTTP request line khatam karne ke liye, aur doosri (blank line) ye signal dene ke liye ke request headers/body khatam ho gaye hain aur server ab response bhej sakta hai — ye HTTP protocol ki requirement hai.
12. Jab specified host/port pe koi service listen nahi kar rahi ho, ya network down ho, ya server hi down ho — tab `connect()` fail (blow up) ho sakti hai.
13. (1) Socket banao, (2) Server se connect karo, (3) GET request encode karke bhejo, (4) Loop mein chunks (512 bytes) receive karo jab tak connection close na ho, (5) Har chunk decode karke print karo, (6) Socket close karo.